

An example logging configuration within the `postgresql.conf` file could look as follows:

```
logging_collector = on
log_destination   = 'stderr,csvlog,jsonlog'
log_directory     = 'log'
log_filename      = 'postgresql-%Y-%m-%d.log'
log_rotation_age  = '1d'
log_rotation_size = '50MB'
```

With the preceding settings, the cluster will produce a new log file for each day (or 50 MB of information) within the log directory (relative to PGDATA) using the logging collector, and every log file will have the indication of the year, month, and day it was created. Note that, as an example, the system will produce logs in textual, JSON, and CSV formats simultaneously; PostgreSQL will smartly change the log filename extension to `.json` and `.csv` for the latter two formats respectively.

With the above logging configuration (also used in the chapter's Docker image), inspecting the log directory will produce an output similar to the following one:

```
$ ls -l /postgres/16/data/log
postgresql-2023-07-19.csv
postgresql-2023-07-19.json
postgresql-2023-07-19.log
...
```

The `.log` file is the one with plain textual format, while the other two files have CSV and JSON format entries.

## When to log

It is important to decide when an event must be reported in the logs. There are a lot of options to control the triggering of a log action, specified by means of a threshold. The logging threshold can assume a mnemonic value that indicates the minimum value over which the log event will be inserted into the logs.

The most common values are, in order, `info`, `notice`, `warning`, `error`, `log`, `fatal`, and `panic`, with `info` being the minimum and `fatal` being the highest value.

As an example, if you decide that `warning` is the threshold you want to accept as a minimum, every log event with a lower threshold (such as `info` and `notice`) will not be inserted into the logs.

As you can see, the threshold increases as it moves toward error values such as `fatal` and `panic`, which are always logged automatically because they represent unrecoverable problems. There are also the lowest levels named `debug1` through `debug5` to get development information and inner details about the process executions (that is, they are usually used when developing with PostgreSQL).

The cluster will therefore produce different log events at different times, and all with different levels of priority, which in turn will be inserted into the logs depending on the threshold you have configured.

In particular, there are two parameters that can be used to tune the log threshold: `log_min_messages` and `client_min_messages`.

The former, `log_min_messages`, decides the threshold of the logging system, while the latter decides the threshold of every new user connection. How are they different?

`log_min_messages` specifies what the cluster has to insert into the logs without any regard for incoming user connections, nor their settings. `client_min_messages` decides which log events the client has to report to the user during the connection. Both these settings can assume a value from the preceding list of thresholds.

A typical use case of a development or test environment could be the following:

```
log_min_messages      = 'info'
client_min_messages    = 'debug1'
```

With the preceding configuration, the cluster will log only `info` messages in the textual logs, which is something related to the normal execution of the processes, while incoming user connections will report more detailed messages such as development ones back to the user.

Setting thresholds is not the only way you can decide when to trigger log insertion: there are another couple of settings that can be used to take care of the duration of statements and utilities.

If you are interested in logging statements (i.e., queries) executed by your clients, you have the following logging parameters to tune:

- `log_min_duration_statement` holds an integer value that represents a number of milliseconds. Every statement taking more time than the set value will be logged. Therefore, setting this value to 0 means that every statement occurring in the system will be dumped into the logs.

- `log_min_duration_sample` and `log_statement_sample_rate` are parameters that work together. `log_min_duration_sample` handle a number of milliseconds and logs only a sample of statements running for much longer than the value of milliseconds. In other words, it works similarly to `log_min_duration_statement` but instead of logging every statement, it logs only a fraction of them. The fraction of statements to be logged is decided by `log_statement_sample_rate`, which handles a value between 0 and 1.
- `log_transaction_sample_rate` is a value between 0 and 1 that indicates how many transactions will be fully logged (i.e., every statement of the transaction will appear in the logs) regardless of the statement durations.

The idea behind the sample parameters is to reduce the amount of logging activity (and size), still providing a useful insight on what is happening in the cluster.

In order to better understand the above parameters, consider the following configuration:

```
log_min_duration_statement = 500
log_min_duration_sample = 100
log_statement_sample_rate = 0.8
log_transaction_sample_rate = 0.5
```

The above configuration will log every statement that runs for more than 500 milliseconds (`log_min_duration_statement`) and 80% of every statement taking longer than 100 milliseconds (`log_min_duration_sample` and `log_statement_sample_rate`). Finally, it will log every transaction out of two (`log_transaction_sample_rate`).

You can test the above with the following simple workload:

```
forumbd=> BEGIN;
BEGIN
forumbd=*> SELECT 'transaction 1';
?column?
-----
transaction 1
(1 row)

forumbd=*> ROLLBACK;
ROLLBACK
forumbd=> BEGIN;
```

```

BEGIN
forumdb=*> SELECT 'transaction 2';
      ?column?
-----
transaction 2
(1 row)

forumdb=*> ROLLBACK;
ROLLBACK
forumdb=> SELECT pg_sleep( 2 );
pg_sleep
-----

(1 row)
forumdb=> BEGIN;
BEGIN
forumdb=*> SELECT pg_sleep( 0.120 ); --repeat 10 times
pg_sleep
-----

(1 row)
forumdb=*> ROLLBACK;
ROLLBACK

```

In the logs, you will find something like the following:

```

LOG:  duration: 0.047 ms  statement: BEGIN;
LOG:  duration: 0.253 ms  statement: SELECT 'transaction 2';
LOG:  duration: 0.068 ms  statement: ROLLBACK;
LOG:  duration: 2003.742 ms  statement: SELECT pg_sleep( 2 );
LOG:  duration: 121.593 ms  statement: SELECT pg_sleep( 0.120 );
LOG:  duration: 121.464 ms  statement: SELECT pg_sleep( 0.120 );
LOG:  duration: 120.459 ms  statement: SELECT pg_sleep( 0.120 );
LOG:  duration: 121.448 ms  statement: SELECT pg_sleep( 0.120 );
LOG:  duration: 121.455 ms  statement: SELECT pg_sleep( 0.120 );
LOG:  duration: 121.416 ms  statement: SELECT pg_sleep( 0.120 );

```

Only one transaction of the two has been logged, according to the `log_transaction_sample` parameter. Also note that the `pg_sleep( 2 )` has been inserted because it takes longer than 500 milliseconds (`log_min_duration_statement`), along with 6 out of 10 calls to `pg_sleep( 0.120 )` inserted because `log_transaction_sample_rate` is set to 0.8 (i.e., 80% of running transactions). You can note how the `log_transaction_sample_rate` is not an exact value: even if the configuration tells PostgreSQL to log 80% of the queries, the system has logged less (60%).

## What to log

The quality of the information to log is configured with a rich set of parameters, usually booleans to tune a particular event to log on or off.

One very used and abused setting is `log_statement`: if turned on, it will log every statement executed against the cluster from every connection. This can be very useful because it allows you to reconstruct exactly what the database did and with which statements, but on the other hand, it can also be very dangerous. Logging every statement could make private or sensitive data available in the logs, which could, therefore, become available to unauthorized people. Moreover, logging all the statements could quickly fill up the log storage, in particular, if the cluster is under a heavy load and high concurrency.



Usually, it is much more useful to configure the `log_min_duration_statement` setting to log only “slow” statements, instead of logging them all.

It is possible to fine-tune the category of statements to log via `log_statement`: the setting can have the value of `off`, `ddl`, `mod`, or `all`. It is quite easy to understand what `off` and `all` mean, but `ddl` means that all data definition language statements (for example, `CREATE TABLE`, `ALTER TABLE`, and so on) are logged, while `mod` means that all data manipulation statements (for example, `INSERT`, `UPDATE`, and `DELETE`) are logged. Log categories are each a superset of the previous one, so `mod` also includes `ddl`, while `all` includes the previous and allows also for logging of `SELECT` type statements. It is worth noting that if a statement contains syntax errors, it will not be logged via `log_statement`, no matter what the setting is.

The quality of the information in the log is also established by the `log_line_prefix` parameter. `log_line_prefix` is a pattern string that defines what to insert at the beginning of every log line, and therefore can be used to detail the event that is logged. The pattern is created with a few placeholders in the same way as `sprintf(3)`, and documenting every option here is out of the scope of the book.

Suffice to say that the most useful and common placeholders are as follows:

- `%a` represents the application name (for example, `psql`).
- `%u` represents the username connected to the cluster (role name).
- `%d` is the database where the event happened.
- `%p` is the operating system **process identifier (PID)**.
- `%h` represents the remote host from which the connection to the cluster has been established.
- `%l` is the session line number, an autoincrement counter that helps us to understand the ordering of every statement executed in an interactive session.
- `%t` is the timestamp at which the event happened.

For example, the following configuration will produce a log line that begins with the timestamp of the event, followed by the process identifier of the backend process, then the counter of the command within the session, and then the user, database, and application used to connect to the cluster from the remote host:

```
log_line_prefix = '%t [%p]: [%l] user=%u,db=%d,app=%a,client=%h '
```

The end result of the preceding configuration will be something like the following log line:

```
[3] user=forum,db=forumdb,app=psql,client=[local]LOG:  duration: 3004.132
ms  statement: select pg_sleep( 3 );
```

Thanks to the `log_line_prefix`, it is possible to insert in every log entry information about the user and database the event is related to, and this can help you better understand and analyze what happened in the cluster.

There are also a few special events that can trigger a log insertion, and that are configured by means of the following parameters:

- `log_connections` and `log_disconnections`: These boolean values dictate if PostgreSQL has to insert an entry in the logs every time a user connection is opened or closed.
- `log_checkpoints`: This boolean setting tells PostgreSQL to log information about checkpoints (see *Chapter 11, Transactions, MVCC, WALs, and Checkpoints*, for more details).
- `log_temp_files`: This parameter accepts an integer value that holds a size expressed in kilobytes. Every time PostgreSQL creates a temporary file bigger than the expressed size, a log entry is produced. Therefore, setting this parameter to 0 means that every time PostgreSQL is using a temporary file, a log entry will be appended.

- `log_lock_waits`: This boolean parameter indicates that a log entry should be created every time a user session is waiting too long to acquire a lock. The threshold is the configuration parameter `deadlock_timeout`.

Now that we have learned all about logging, we will move on to extracting information from the logs that are created, using a special tool called pgBadger.

## Extracting information from logs – pgBadger

Thanks to the rich set of information that can be included in the logs, it is possible to automate log information analysis and extraction. There are several tools with this aim, and one of the most popular and complete is pgBadger.

pgBadger is a self-contained Perl 5 application that carefully reads and extracts information from PostgreSQL logs, producing a web dashboard with a summary of all the information it has found in the logs. The aim of this application is to provide you with more useful insights into the logs without having to manually search for specific information.

Using pgBadger is not mandatory; your cluster will work fine without it and you will be able to seek information and problems in the logs regardless. However, using pgBadger provides you with more useful hints about what your server has done.

It is important to note that using pgBadger, as well as performing any automated or manual log analysis, does not provide real-time information, but rather, a look into server activities in the past.

In the following subsections, you will learn how to install and use pgBadger.

### Installing pgBadger

pgBadger requires Perl 5 to be installed on the system it will run on, and that is the only dependency it has. You can run pgBadger on the same host the PostgreSQL cluster is running on, or on a remote system (as will be shown in a later subsection). In this section, we will assume pgBadger will be installed and executed on the very same machine the PostgreSQL cluster is running on.

The easiest way to install pgBadger is by means of the operating system package manager, such as the following on GNU/Debian and Ubuntu-based systems:

```
$ sudo apt install pgbadger
```

It is also possible to install pgBadger from the source with the following steps:

```
$ wget https://github.com/darold/pgbadger/archive/v12.0.tar.gz
$ tar xzvf v12.0.tar.gz
$ cd pgbadger-12.0
$ perl Makefile.PL
$ make
$ sudo make install
```

Once you have installed pgBadger, you can test that it is working by typing:

```
$ pgbadger --version
pgBadger version 12.0
```

If the program replies with the version number, everything should be fine and ready to be used.



pgBadger is already installed in the Docker image for this chapter.

## Configuring PostgreSQL logging for pgBadger usage

pgBadger is smart enough to be able to understand PostgreSQL logs in many cases, but there are some circumstances where you need to specify some configuration options to make PostgreSQL produce more understandable logs.

First of all, pgBadger needs to have access to the PostgreSQL logs, and this means you should use `logging_collector` to produce the logs. If you change `log_line_prefix`, you should pass the same configuration setting to pgBadger, so that it is able to correctly parse the log prefix. Last, you should enable as many logging contexts as possible.

The following is an example of the configuration parameters that make PostgreSQL produce logs that pgBadger can understand correctly:

```
logging_collector = on
log_destination   = 'stderr,csvlog,jsonlog'
log_directory     = 'log'
log_filename      = 'postgresql-%Y-%m-%d.log'
log_rotation_age  = '1d'
log_rotation_size = '50MB'
```

```
log_min_duration_statement = 500  
log_min_duration_sample = 100  
log_statement_sample_rate = 0.8  
log_transaction_sample_rate = 0.5  
  
log_min_duration_statement = 0
```

Once the server has been configured to get the new log configuration, you can start using pgBadger.

## Using pgBadger

Once PostgreSQL has begun producing logs, you can analyze the results with pgBadger. Before you run pgBadger, especially on a test system, you should generate (or wait for) some traffic and statements (as well as transactions), or the produced dashboard will be empty.

Before starting to use pgBadger, it is appropriate to create a location to store the reports and all the related stuff. This is not mandatory, but simplifies the maintenance and archiving of reports later on when you may need to keep them. Let's create a directory, and let's assign the same Postgres user that runs the cluster the ownership of the directory (again, this is not mandatory but simplifies the workflow a little):

```
$ sudo mkdir /data/html  
$ sudo chown postgres:www-data /data/html
```

It is now time to launch pgBadger for the first time:

```
$ pgbadger -o /data/html/first_report.html \
          /postgres/16/data/log/postgresql
-2023-07-19.log
[=====>] Parsed 261891612 bytes of 261891612 (100.00%),
queries: 1428472, events: 2
7
LOG: Ok, generating html report...
```

The first argument, `-o`, specifies the name of the file where we want the report to be stored. pgBadger produces exactly one file for every run, so you need to change the filename if you want to generate another report without overwriting an existing report.

The second argument is the PostgreSQL log file to analyze; you can also specify JSON or CSV files and pgBadger will parse them accordingly.

The program runs for a few seconds, or minutes depending on the size of the log file, and reports some statistical information about what it found on the log file (in this example, 1.4 million statements). You can check the generated report file quite easily:



If you are going to analyze big log files, or many of them, you can use the parallel mode of pgBadger with the `-j` option followed by the number of parallel processes to spawn. For example, passing `-j 4` means that every log file will be divided into four parts, each one analyzed by a single process. Thanks to parallelism, you can exploit all the cores of your machine and get results faster for a large amount of logs.

```
$ ls -ls /data/html/first_report.html
1172 /data/html/first_report.html
```